



# Flyway basics and configuration

# Overview



Flyway is an open-source database migration tool. It strongly favors simplicity and convention over configuration.

It is based around just 7 basic commands: Migrate, Clean, Info, Validate, Undo, Baseline and Repair.

Migrations can be written in SQL (database-specific syntax (such as PL/SQL, T-SQL, ...) is supported) or Java (for advanced data transformations or dealing with LOBs).

It has a Command-line client. If you are on the JVM, we recommend using the Java API (also works on Android) for migrating the database on application startup. Alternatively, you can also use the Maven plugin or Gradle plugin.

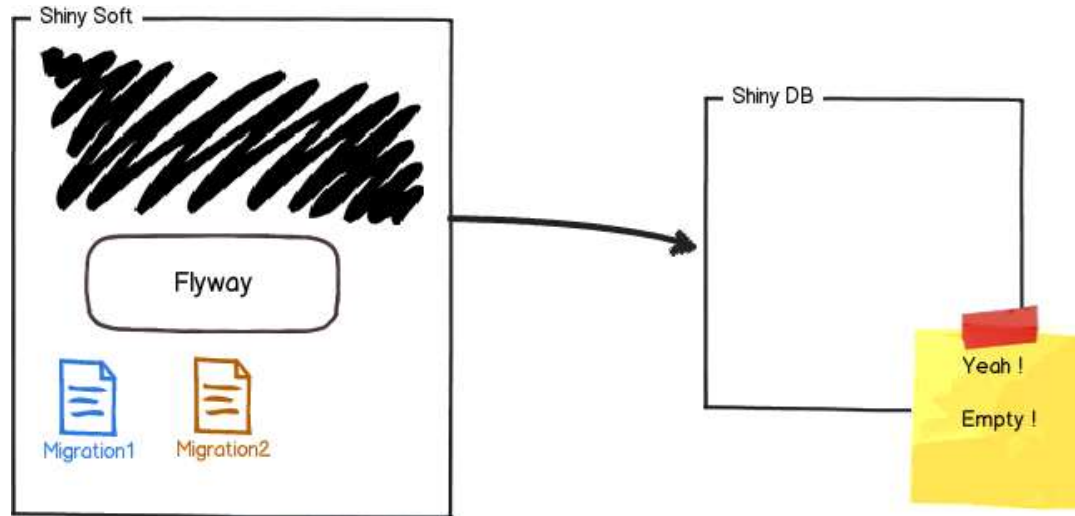
# Overview

And if that's not enough, there are plugins available for Spring Boot, Dropwizard, Grails, Play, SBT, Ant, Griffon, Grunt, Ninja and more!

Supported databases are Oracle, SQL Server (including Amazon RDS and Azure SQL Database), Azure Synapse (Formerly Data Warehouse), DB2, MySQL (including Amazon RDS, Azure Database & Google Cloud SQL), Aurora MySQL, MariaDB, Percona XtraDB Cluster, TestContainers, PostgreSQL (including Amazon RDS, Azure Database, Google Cloud SQL & Heroku), Aurora PostgreSQL, Redshift, CockroachDB, SAP HANA, Sybase ASE, Informix, H2, HSQLDB, Derby, Snowflake, SQLite and Firebird.

# How Flyway works

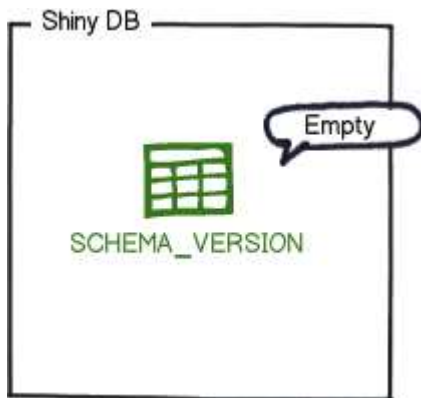
The easiest scenario is when you point Flyway to an empty database.



# How Flyway works

It will try to locate its **schema history table**. As the database is empty, Flyway won't find it and will **create** it instead.

You now have a database with a single empty table called flyway\_schema\_history by default:

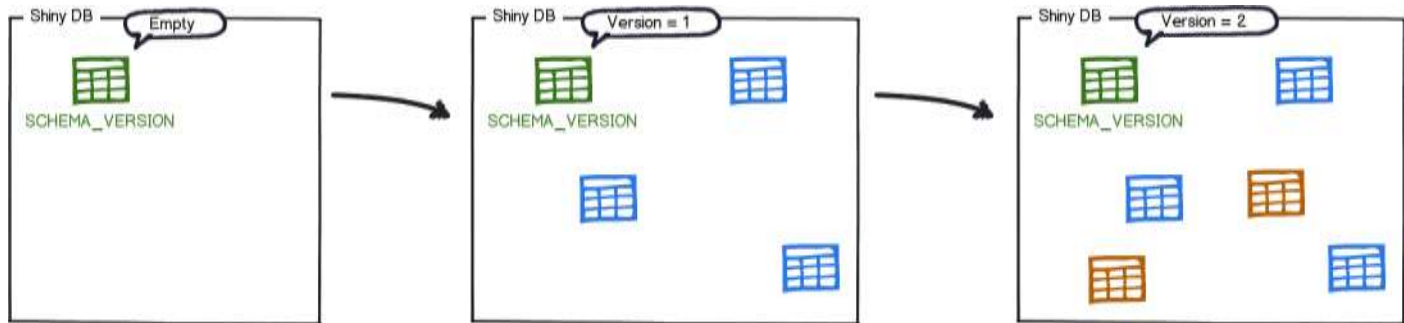


# How Flyway works

This table will be used to track the state of the database.

Immediately afterwards Flyway will begin **scanning** the filesystem or the classpath of the application for **migrations**. They can be written in either Sql or Java.

The migrations are then **sorted** based on their **version number** and applied in order:



# How Flyway works

As each migration gets applied, the schema history table is updated accordingly:

flyway\_schema\_history

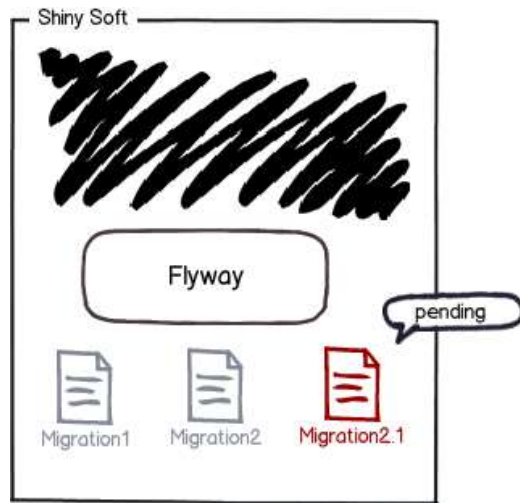
| installed_rank | version | description   | type | script                | checksum   | installed_by | installed_on          | execution_time | success |
|----------------|---------|---------------|------|-----------------------|------------|--------------|-----------------------|----------------|---------|
| 1              | 1       | Initial Setup | SQL  | V1__Initial_Setup.sql | 1996767037 | axel         | 2016-02-04 22:23:00.0 | 546            | true    |
| 2              | 2       | First Changes | SQL  | V2__First_Changes.sql | 1279644856 | axel         | 2016-02-06 09:18:00.0 | 127            | true    |

With the metadata and the initial state in place, we can now talk about **migrating to newer versions**.

Flyway will once again scan the filesystem or the classpath of the application for migrations. The migrations are checked against the schema history table. If their version number is lower or equal to the one of the version marked as current, they are ignored.

# How Flyway works

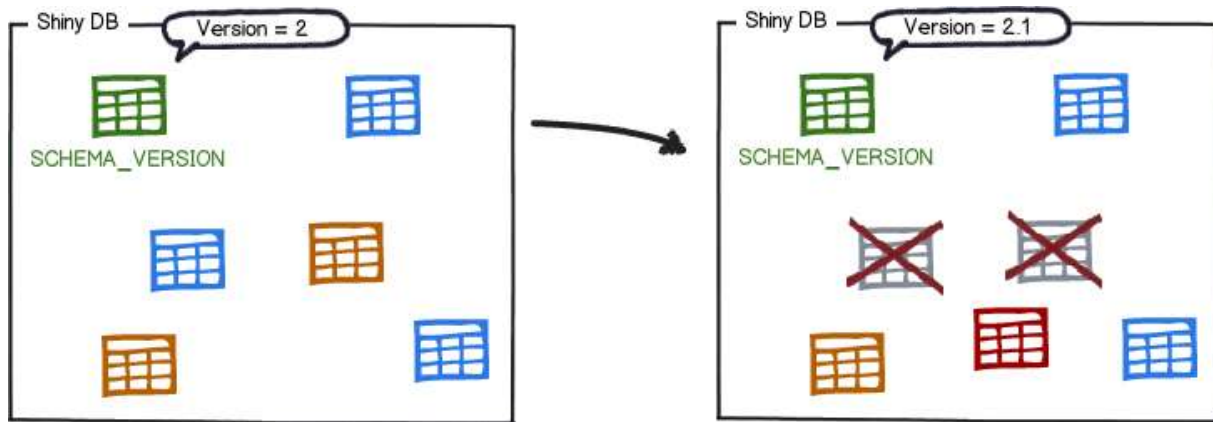
The remaining migrations are the **pending migrations**: available, but not applied.





# How Flyway works

They are then **sorted by version number** and **executed in order**:



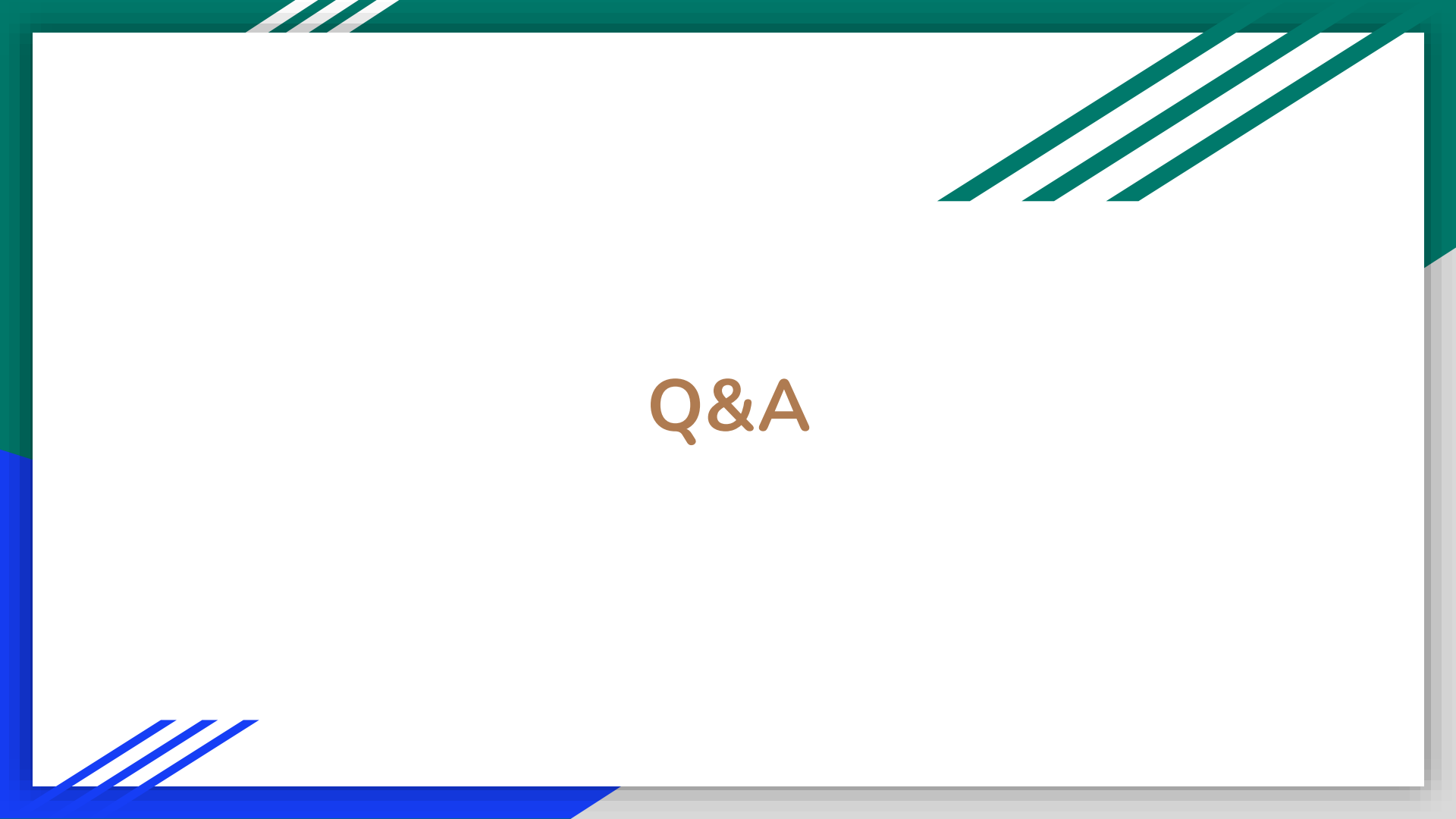
# How Flyway works

The **schema history table** is **updated** accordingly:

`flyway_schema_history`

| installed_rank | version | description   | type | script                | checksum   | installed_by | installed_on          | execution_time | success |
|----------------|---------|---------------|------|-----------------------|------------|--------------|-----------------------|----------------|---------|
| 1              | 1       | Initial Setup | SQL  | V1__Initial_Setup.sql | 1996767037 | axel         | 2016-02-04 22:23:00.0 | 546            | true    |
| 2              | 2       | First Changes | SQL  | V2__First_Changes.sql | 1279644856 | axel         | 2016-02-06 09:18:00.0 | 127            | true    |
| 3              | 2.1     | Refactoring   | JDBC | V2.1__Refactoring     |            | axel         | 2016-02-10 17:45:05.4 | 251            | true    |

**And that's it!** Every time the need to evolve the database arises, whether structure (DDL) or reference data (DML), simply create a new migration with a version number higher than the current one. The next time Flyway starts, it will find it and upgrade the database accordingly.



Q&A